Chapter 12: Binary Search Trees

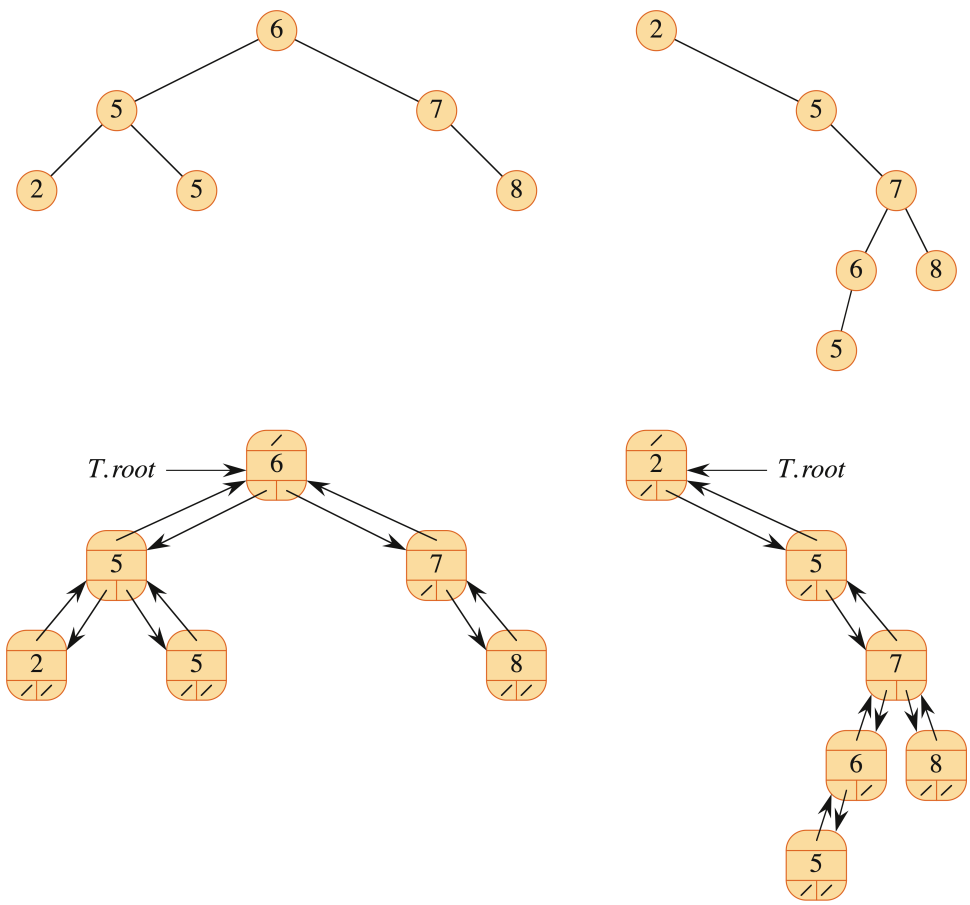
What is a Binary Search Tree (BST)?

A binary search tree is a binary tree with the following two properties:

1. If a node y is in the left subtree of a node x, then $y.key \leq x.key$.
2. If a node y is in the right subtree of a node x, then $y.key \geq x.key$.

The above definition allows nodes with equal keys to be present in either subtree of a node in a BST. In practice, we will have nodes with equal keys be only in the right subtree of a node to simplify the code.

Java: For more general objects implementing the `Comparable` interface, use $y.compareTo(x)$



(a) (b)

Inorder tree walk can print out all the keys in a binary search tree in a sorted order.

```
INORDER-TREE-WALK(x)
if x != NIL
    INORDER-TREE-WALK(x.left)
    print x
    INORDER-TREE-WALK(x.right)
```

We can similarly do a **preorder** and **postorder** walk or traversal. In preorder traversal, print the object in the node before traversing left and right subtrees. In postorder traversal, print the object in the node after traversing left and right subtrees. All of them take worst-case runtime of $\Theta(n)$.

Recommended Exercises: 12.1-1, 12.1-2, 12.1-4. *Challenging:* 12.1-3 (solved on next page)

Other Recommended Exercises:

1. Write a recursive procedure for counting the number of nodes in a binary search tree.

   Solution:

   ```
   COUNT-NODES(x)
   if x == NIL
       return 0
   count = 1
   count = count + COUNT-NODES(x.left)
   count = count + COUNT-NODES(x.right)
   return count
   ```

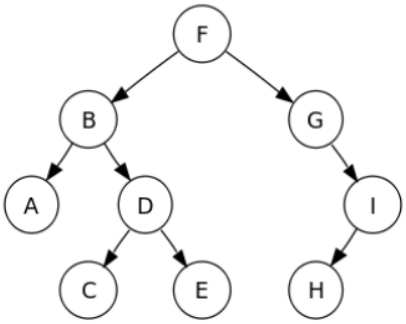
2. Write a recursive procedure for finding the height of a binary search tree.

You should write the procedure for the height on your own!

Solution 12.1-3:

```
ITERATIVE-INORDER-TREE-WALK(x)
initialize an empty stack S
while true
   //traverse to the leftmost leaf
   while x != NIL
      PUSH(S, x)
      x = x.left
   //if stack is empty, then we are done
   if EMPTY-STACK(S)
       return
   //pop the top element, print it and then add nodes
   //in the right subtree to the stack
   x = POP(S)
   print x.data
   x = x.right
```

Example:



Step	Output	Stack
0		F,B,A
1	A	F,B
2	B	F,D,C
3	C	F,D
4	D	F,E
5	E	F
6	F	G
7	G	I,H
8	H	I
9	I	

Querying a Binary Search Tree

Querying a Binary Search Tree: retrieve information from the tree without modifying the tree.

Query operations of a binary search tree include Search, Minimum, Maximum, Successor, Predecessor, ...

All of the above operations take $O(h)$, where h is the height of the tree.

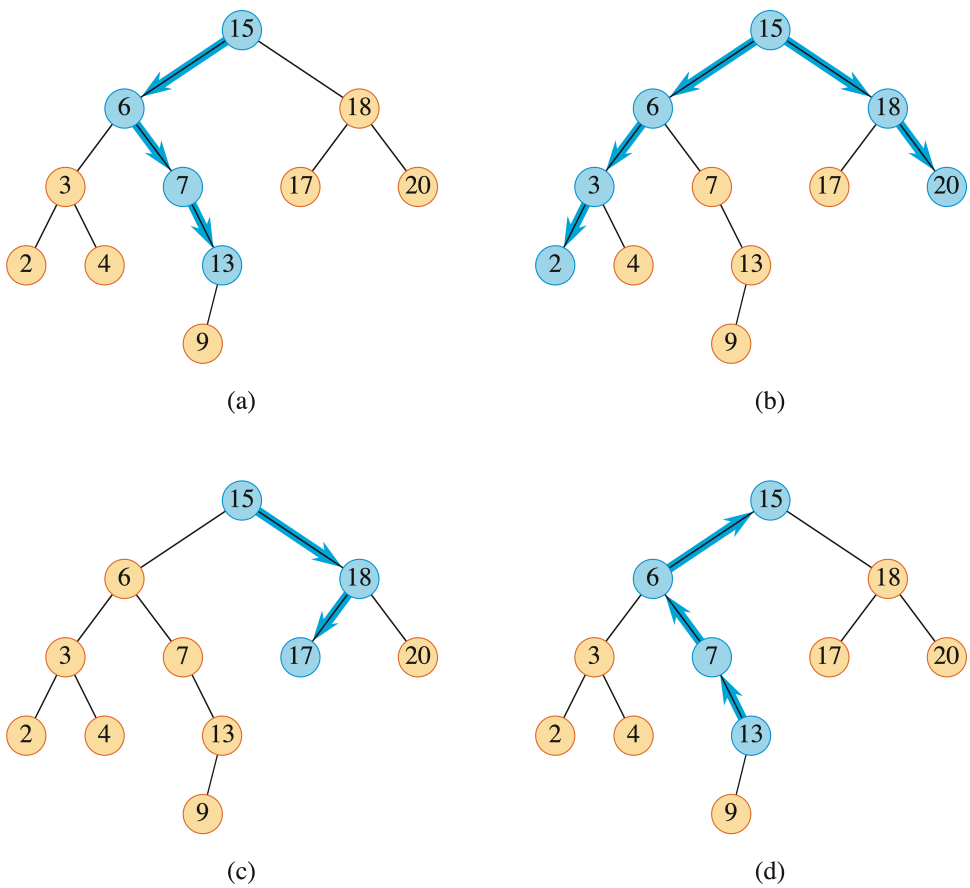


Figure (a) shows a search for key=13, (b) shows search for the min and max, (c) shows the search for the successor of 15, and (d) shows the search for the successor of 13.

```
TREE-SEARCH(x, k)
if x == NIL or k == x.key
    return x
if k < x.key
      return TREE-SEARCH(x.left, k)
else return TREE-SEARCH(x.right, k)
```

The nodes searched during the recursion form a path from the root downward. Thus, running time is $O(h)$.

```
ITERATIVE-TREE-SEARCH(x, k)
while x != NIL and k != x.key
      if k < x.key
           x = x.left
      else x = x.right
return x
```

```
TREE-MINIMUM(x)
while x.left != NIL
     x = x.left
return x
```

```
TREE-MAXIMUM(x)
while x.right != NIL
     x = x.right
return x
```

The successor of a node x is the node y, where

$$y = \begin{cases} \text{Minimum}(x.right) & \text{if } x.right \neq \text{NIL} \\ \text{The lowest ancestor of } x \text{ whose left child is also an ancestor of } x & \text{if } x.right = \text{NIL} \end{cases}$$

```
TREE-SUCCESSOR(x)
if x.right != NIL
   return TREE-MINIMUM(x.right)
else
   y = x.p
   while y != NIL and x == y.right
      x = y
      y = y.p
   return y
```

The worst-case runtime is $O(h)$, since we either follow a path downward (1st case) or a path upward (2nd case)

Recommended Exercises: 12.2-1, 12.2-2, 12.2-4, 12.2-5, 12.2-6.

Solve in class: 12.2-3.

Solution to Exercise 12.2-3: The predecessor of a node x is the node y, where

$$y = \begin{cases} \text{Maximum}(x.left) & \text{if } x.left \neq \text{NIL} \\ \text{The lowest ancestor of } x \text{ whose right child is also an ancestor of } x & \text{if } x.left = \text{NIL} \end{cases}$$

```
TREE-PREDECESSOR(x)
if x.left != NIL
    return TREE-MAXIMUM(x.left)
else
    y = x.p
    while y != NIL and x == y.left
        x = y
        y = y.p
    return y
```

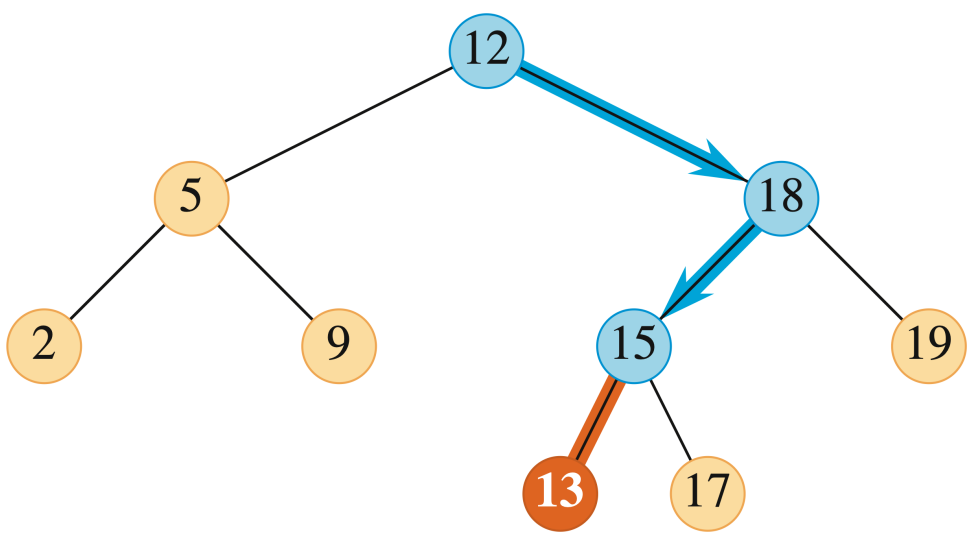
The worst-case runtime is $O(h)$, using same argument as for TREE-SUCCESSOR(x)

Insertion and Deletion

Insertion

TREE-INSERT(T, z): insert a node z into a binary search tree T, where $z.key = v$, $z.left = z.right = z.p = NIL$ (initially, but the pointers may change during the process of insertion)

TREE-INSERT always inserts a new node z as a leaf node.



```
TREE-INSERT(T, z)
// Insert node z, where z.key = v, z.left = NIL z.right = NIL
1.  x = T.root            // x keeps track of the path for insertion
2.  y = NIL               // y will track the parent of x
3.  while x != NIL
4.        y = x
5.        if z.key < x.key
6.             x = x.left
7.        else x = x.right
8.  z.p = y
9.  if y == NIL
10.        T.root = z
11. elseif z.key < y.key
12.           y.left = z
13. else y.right = z
```

Steps 3 - 7: find the position to insert the new node.

Steps 8 - 13: set the pointers to insert the new node.

Takes $O(h)$ worst-case runtime: trace downward from the root to a leaf to find the position to insert.

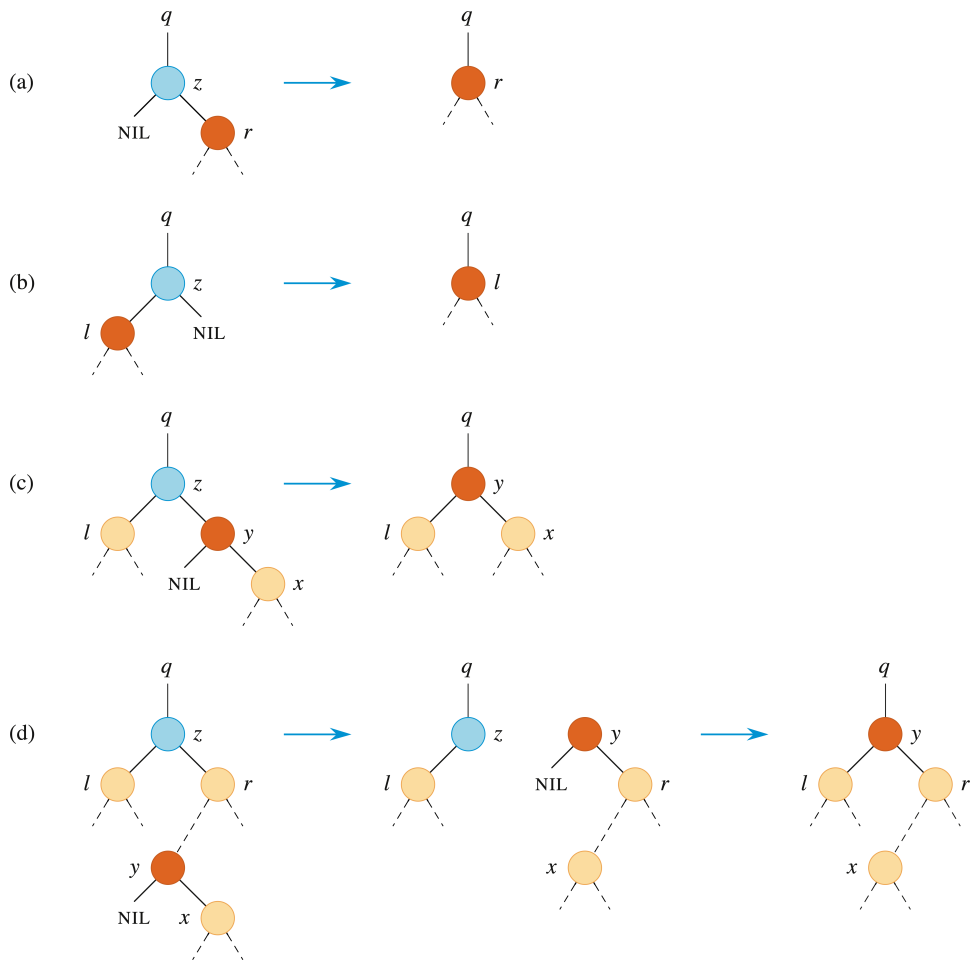
Give an example, using the animation in the references.

Questions:

- What kind of BST do we get if we insert n elements that are already sorted in **ascending** order? What is its height?
- What kind of BST do we get if we insert n elements that are already sorted in **descending** order? What is its height?

Deletion

To delete a node z from a binary search tree, there are 4 cases to consider.



Case (a) If z has no left child, then replace z by its right child, which may or may not be NIL. This corresponds to part (a) in the figure.

Case (b) Otherwise, z has just a left child, we replace z by its left child. This corresponds to part (b) in the figure.

Case (c) and (d) Otherwise, z has both a left child and a right child. Find z's successor y, which lies in z's right subtree and has no left child. Splice node y out of its current location and replace z by y in the tree. How to do depends on the whether y is z's right child. This leads to the last two cases.

Case (c) If y is z's right child, then as in part (c) of the figure, replace z by y, leaving y's right child alone.

Case (d) Otherwise, y lies within z's right subtree but is not z's right child. In this case, as in part (d) of the figure, first replace y by its own right child, and then replace z by y.

```
TRANSPLANT(T, u, v)
//replace the subtree rooted at node u with the subtree rooted at node v
if u.p == NIL
      T.root = v
elseif u == u.p.left
      u.p.left = v
else u.p.right = v
if v != NIL
     v.p = u.p
```

```
TREE-DELETE(T, z)
if z.left == NIL
     TRANSPLANT(T, z, z.right)
elseif z.right == NIL
     TRANSPLANT(T, z, z.left)
else y = TREE-MINIMUM(z.right)
      if y != z.right
         TRANSPLANT(T, y, y.right)
         y.right = z.right
         y.right.p = y
      TRANSPLANT(T, z, y)
      y.left = z.left
      y.left.p = y
```

Takes $O(h)$ worst-case runtime: case (a) or (b) take $\Theta(1)$, but case (c) and (d) takes $O(h)$.

Recommended Exercises: 12.3-1, 12.3-2, 12.3-3, 12.3-4, 12.3-5.

Solve in class: 12.3-3, 12.3-5.

Exercise 12.3-5: Is deletion commutative, in the senses that deleting x followed y gives us the same tree if we delete y followed by x? Argue why it is or give an counterexample.

Try deleting 5,3 and 3,5

```
 5
/ \
3  7
  /
 6
```

Randomly built binary search tree have an expected height of $O(\lg n)$, similar to the best-case. See below for an implementation (one simple, one generic) that runs an experiment to determine average height of randomly created binary search trees. This is an example of performance testing!

- See a simple implementation of a BST here: https://github.com/BoiseState/CS321-resources/tree/master/examples/binary-search-tree
- See a generic implementation of a BST here: https://github.com/BoiseState/CS321-resources/tree/master/examples/binary-search-tree-generic

But what if we have an unbalanced binary search tree (because it wasn't random...). How can we rebalance an existing binary search tree?

Recommended Exercise: We can do an inorder walk and then recursively build back a balanced tree. Develop pseudo-code for this procedure.

References

- TREE visualization and interactive explorer: https://visualgo.net/en/bst

Height Balanced Search Trees

Red-Black trees are binary search trees with an extra bit of storage per node: its color, which can be either red or black. By constraining the way nodes are colored along any path from the root to a leaf, red-black trees ensure that no path is more than twice as long as any other, thus maintaining a balanced tree.

To see how red-black trees height balance, play with this visualization: https://www.cs.usfca.edu/~galles/visualization/RedBlack.html

Another example of a height balanced search tree is **AVL tree**, which maintains the balance by keeping track of the height of each node and performing rotations to maintain the balance after each insert or delete.

B-trees are a generalization of binary search trees in which a node can have more than two children. B-trees are optimized for systems that read and write large blocks of data, such as databases and file systems. We will study them in the next module.